

Sorting Algorithms Reference (Part 1)

1. Bubble Sort

```
#include <iostream>

using namespace std;

int main(){

    int n;

    cin>>n;

    int a[n];

    for(int i=0;i<n;i++){
        cin>>a[i];
    }

    for(int i=0;i<n-1;i++){
        for(int j=0;j<n-i-1;j++){
            if(a[j]>a[j+1])
                swap(a[j],a[j+1]);
        }
    }

    for(int i=0;i<n;i++)
        cout<<a[i]<<" ";

    return 0;
}
```

2. Selection Sort

```
#include <iostream>

using namespace std;

int main(){

    int n;

    cin>>n;

    int a[n];

    for(int i=0;i<n;i++){
        cin>>a[i];
    }

    for(int i=0;i<n-1;i++){
        int mn=i;

        for(int j=i+1;j<n;j++){
            if(a[j]<a[mn])
                mn=j;
        }

        swap(a[i],a[mn]);
    }

    for(int i=0;i<n;i++)
        cout<<a[i]<<" ";

    return 0;
}
```

Sorting Algorithms Reference (Part 2)

3. Merge Sort (Vector)

```
#include <iostream>

#include <vector>

using namespace std;

void mergeSort(vector<int> &a){
    if(a.size()<=1)
        return;

    int mid = a.size()/2;

    vector<int> left (a.begin(),a.begin()+mid);
    vector<int> right (a.begin()+mid,a.end());

    mergeSort(left);
    mergeSort(right);

    int i=0, j=0, k=0;

    while(i<left.size() && j<right.size()){
        if(left[i]<=right[j])
            a[k++]=left[i++];
        else
            a[k++]=right[j++];
    }

    while(i<left.size())
        a[k++]=left[i++];

    while(j<right.size())
        a[k++]=right[j++];
    }

int main(){
    int n;

    cin>>n;

    vector<int> a(n);

    for(int i=0;i<n;i++)
        cin>>a[i];

    mergeSort(a);

    for(int x:a)
        cout<<x<<" ";

    return 0;
}
```

4. Quick Sort (Vector)

```
#include <iostream>

#include <vector>

using namespace std;

void quickSort(vector<int> &a, int l,int h){
    if(l>=h)
        return;

    int pivot = a[h];

    int i=l-1;

    for(int j=l;j<h;j++){
        if(a[j]<pivot) {
            i++;
            swap(a[i],a[j]);
        }
    }

    swap(a[i+1],a[h]);

    int p=i+1;

    quickSort(a,l,p-1);
    quickSort(a,p+1,h);
    }

int main(){
    int n;

    cin>>n;

    vector<int> a(n);

    for(int i=0;i<n;i++)
        cin>>a[i];

    quickSort(a,0,n-1);

    for(int x:a)
        cout<<x<<" ";

    return 0;
}
```

Descending Order & Quick Reference

Section 1: Descending Output Snippets

If the problem only requires the printed sequence to be in descending order, you can print the sorted structure backwards:

Array Reverse Print

```
for(int i=n-1;i>=0;i--)  
  
cout<<a[i]<<" ";
```

Vector Reverse Print

```
for(int i=a.size()-1;i>=0;i--)  
  
cout<<a[i]<<" ";
```

Section 2: Comparison Modification Table

If the question explicitly demands a descending sorting algorithm implementation, modify only the comparison conditions as shown:

Algorithm	Ascending Operator	Descending Operator
Selection Sort	$a[j] < a[mn]$	$a[j] > a[mn]$
Bubble Sort	$a[j] > a[j+1]$	$a[j] < a[j+1]$
Merge Sort	$left[i] \leq right[j]$	$left[i] \geq right[j]$
Quick Sort	$a[j] < pivot$	$a[j] > pivot$

EXAM DAY REMINDER

- ✓ **Scenario A:** If the question only asks you to print output in descending order, do not rewrite the sorting algorithm logic. Just run the standard ascending code and print the array/vector backwards (as shown in Section 1). This is fast and less prone to errors.
- ✓ **Scenario B:** If the question specifically asks to "Implement [Algorithm] in Descending Order", use the code structure from Page 1 or Page 2, but replace the operator exactly as shown in Section 2.